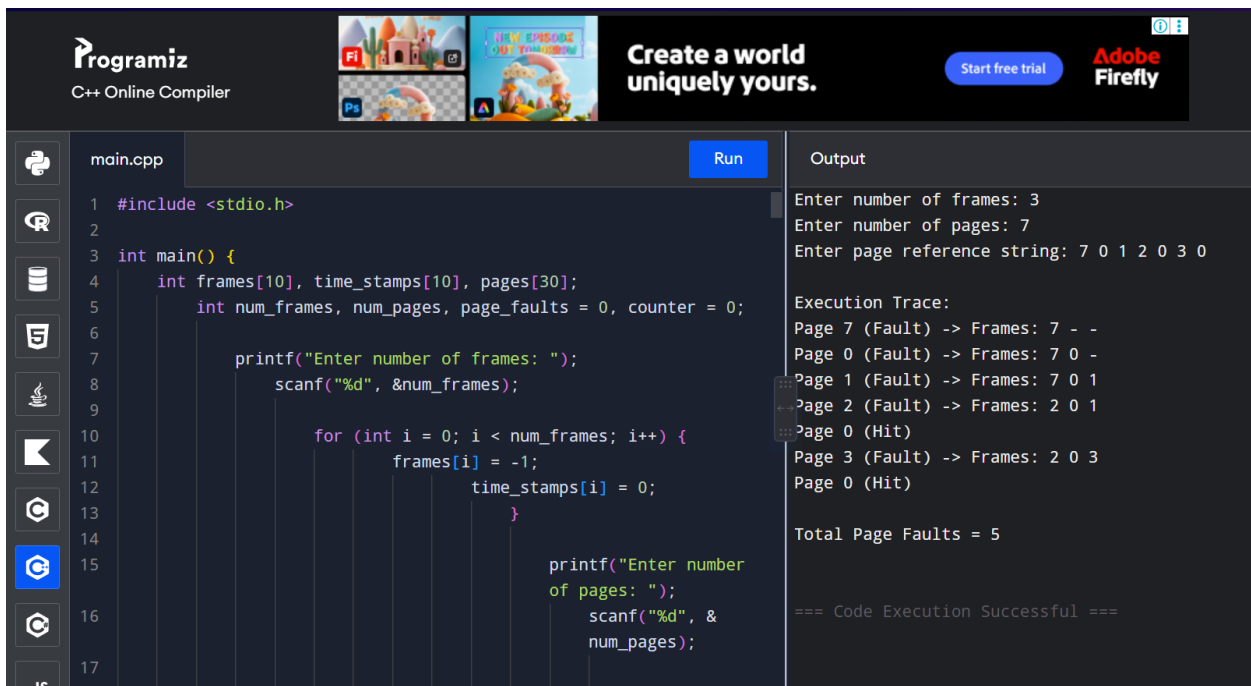


EXPERIMENT 26:

26. Construct a C program to simulate the Least Recently Used paging technique of memory management.

AIM:

To construct a C program to simulate the Least Recently Used (LRU) paging technique of memory management.



The screenshot displays the Programiz C++ Online Compiler interface. The code editor on the left contains a C program for simulating the Least Recently Used (LRU) paging technique. The program prompts the user for the number of frames and pages, then for a page reference string. It initializes an array of frames and time stamps, and then processes the page reference string, outputting the state of frames after each page access and the total number of page faults.

```
1 #include <stdio.h>
2
3 int main() {
4     int frames[10], time_stamps[10], pages[30];
5     int num_frames, num_pages, page_faults = 0, counter = 0;
6
7     printf("Enter number of frames: ");
8     scanf("%d", &num_frames);
9
10    for (int i = 0; i < num_frames; i++) {
11        frames[i] = -1;
12        time_stamps[i] = 0;
13    }
14
15    printf("Enter number of pages: ");
16    scanf("%d", &num_pages);
17
18    printf("Enter page reference string: ");
```

The output window on the right shows the execution results:

```
Enter number of frames: 3
Enter number of pages: 7
Enter page reference string: 7 0 1 2 0 3 0

Execution Trace:
Page 7 (Fault) -> Frames: 7 - -
Page 0 (Fault) -> Frames: 7 0 -
Page 1 (Fault) -> Frames: 7 0 1
Page 2 (Fault) -> Frames: 2 0 1
Page 0 (Hit)
Page 3 (Fault) -> Frames: 2 0 3
Page 0 (Hit)

Total Page Faults = 5

=== Code Execution Successful ===
```

PSEUDO CODE :

BEGIN

PRINT "Enter number of frames:"

READ num_frames

DECLARE frames array of size num_frames

DECLARE time_stamps array of size num_frames

INITIALIZE frames with -1

INITIALIZE time_stamps with 0

```
PRINT "Enter number of pages:"
```

```
READ num_pages
```

```
DECLARE pages array of size num_pages
```

```
PRINT "Enter page reference string:"
```

```
FOR i FROM 0 TO num_pages - 1 DO
```

```
    READ pages[i]
```

```
END FOR
```

```
SET page_faults = 0
```

```
SET counter = 0
```

```
FOR i FROM 0 TO num_pages - 1 DO
```

```
    SET page = pages[i]
```

```
    SET page_found = 0
```

```
    INCREMENT counter
```

```
    // Search if page exists in frame
```

```
    FOR j FROM 0 TO num_frames - 1 DO
```

```
        IF frames[j] == page THEN
```

```
            SET page_found = 1
```

```
            SET time_stamps[j] = counter // Update last used timestamp
```

```
            BREAK
```

```
        END IF
```

```
    END FOR
```

```
IF page_found == 0 THEN
```

```
    SET lru_index = 0
```

```
    SET empty_found = 0
```

```

// Check for empty slot first
FOR j FROM 0 TO num_frames - 1 DO
    IF frames[j] == -1 THEN
        SET lru_index = j
        SET empty_found = 1
        BREAK
    END IF
END FOR

// If no empty slot, find frame with minimum timestamp
IF empty_found == 0 THEN
    SET min_time = time_stamps[0]
    SET lru_index = 0
    FOR j FROM 1 TO num_frames - 1 DO
        IF time_stamps[j] < min_time THEN
            SET min_time = time_stamps[j]
            SET lru_index = j
        END IF
    END FOR
END IF

SET frames[lru_index] = page
SET time_stamps[lru_index] = counter
INCREMENT page_faults

PRINT "Page " + page + " (Fault) -> Frames: " + CURRENT_FRAMES
ELSE
    PRINT "Page " + page + " (Hit)"
END IF
END FOR

```

```
    PRINT "Total Page Faults: " + page_faults  
END
```

CODE:

```
#include <stdio.h>  
  
int main() {  
  
    int frames[10],  
    time_stamps[10],  
    pages[30];  
  
    int num_frames,  
    num_pages, page_faults  
    = 0, counter = 0;  
  
    printf("Enter number  
of frames: ");  
  
    scanf("%d",  
&num_frames);  
  
    for (int i = 0; i <  
num_frames; i++) {
```

```
frames[i] = -1;

time_stamps[i] = 0;

}


printf("Enter number
of pages: ");

scanf("%d",
&num_pages);


printf("Enter page
reference string: ");

for (int i = 0; i <
num_pages; i++) {

    scanf("%d",
&pages[i]);

}


printf("\nExecution
Trace:\n");
```

```
    for (int i = 0; i <
num_pages; i++) {

    int page_found = 0;

    counter++;

    for (int j = 0; j <
num_frames; j++) {

        if (frames[j] ==
pages[i]) {

            page_found =
1;

time_stamps[j] =
counter;

            break;

        }

    }

    if (page_found ==
```

```
0) {  
  
    int lru_index =  
  
    0;  
  
    int empty_found  
= 0;  
  
    for (int j = 0; j <  
num_frames; j++) {  
  
        if (frames[j]  
== -1) {  
  
            lru_index =  
j;  
  
            empty_found = 1;  
  
            break;  
  
        }  
  
    }  
  
    if
```

```

(!empty_found) {

    int min_time
= time_stamps[0];

    lru_index = 0;

    for (int j = 1; j
< num_frames; j++) {

        if
(time_stamps[j] <
min_time) {

            min_time
= time_stamps[j];

            lru_index
= j;

        }

    }

}

frames[lru_index] =
pages[i];

```



```
time_stamps[lru_index]
```

```
= counter;
```

```
    page_faults++;
```

```
    printf("Page %d
```

```
(Fault) -> Frames: ",
```

```
pages[i]);
```

```
    for (int k = 0; k  
< num_frames; k++) {
```

```
        if (frames[k]  
!= -1)
```

```
            printf("%d  
", frames[k]);
```

```
        else
```

```
            printf("- ");
```

```
    }
```

```
    printf("\n");
```

```
    } else {
```

```
        printf("Page %d  
(Hit)\n", pages[i]);  
  
    }  
  
}
```

```
    printf("\nTotal Page  
Faults = %d\n",  
page_faults);  
  
    return 0;  
  
}
```